

INF-1400: Object-oriented programming

Mandatory assignment 3

Jens Christian Valen Leynse

April 19th, 2024

1 Introduction

The assignment was to create a mayhem clone, which is a game where two spaceships try to take each other down, while surviving all obstacles. The code is written in python 3, and the code includes the requirements of the assignment implemented with classes and methods.

1.1 Requirements

1. The two spaceships shall have four controls: rotating left and right, thrust for propulsion, and fire which should shoot a projectile.
2. Minimum one obstacle in the game world.
3. The ships shall be able to collide with obstacles, the walls and each other.
4. A gravity shall affect the ships.
5. Each ship shall have a score that increases and decreases according to if the player shoots the opponent down, or if the player crashes or gets shot down.
6. Each ship shall have a limited amount of fuel. This fuel decreases so that the player will have to refuel at a landing pad or at a refueling object placed on the screen.

2 Technical Background

The classes are all connected through a has-a relationship to send the information around. This is done through instances where a class is called from within another class. It's done once to connect every class, except for one class which is instantiated several time into a list. Three python libraries are included in the code; pygame, random and time.

3 Design

The code is organized into ten classes, with the spaceship class and the game class being the most prominent. These two classes can be categorized into separate groups: those that interact with spaceships and those that don't. While all classes have elements to draw, only the ones interacting affect the player. The size of each object is determined using functions from the Pygame library, allowing for rotation and scaling of images or sprites.

The first group of interacting classes, represented by the object file, includes:

- **Obstacle class:** Displays a giant upside-down thumb that falls through the screen randomly. It is spawned at random positions within certain screen boundaries and is updated periodically to change its position.
- **Landing pad class:** Represents the image of the landing pad or stand and its position on the screen. The image is drawn during the game loop.
- **Fuel sheet class:** Represents the refueling item, typically displayed as a note sheet. It moves randomly across the screen to provide the player with opportunities for refueling.

The second group, represented by the classes file, includes:

- **Explosion class:** Manages the animation for the death sequence. It displays ten images sequentially to create the explosion effect when a spaceship crashes or dies.
- **Score class:** Displays the scores of the two players on the screen corners using text. It retrieves the font from the Pygame library and displays the points earned by each player.
- **Health bar class:** Displays the health of each spaceship using rectangles along a line. If a player's health reaches zero, a Boolean statement triggers destruction of the spaceship.
- **Fuel bar class:** Displays the remaining fuel for each spaceship. If the fuel runs out, the ship's thrust stops.
- **Bullets class:** Represents the projectiles shot from spaceships. It interacts with the spaceships by being shot out from them, with each bullet having its own movement and appearance.

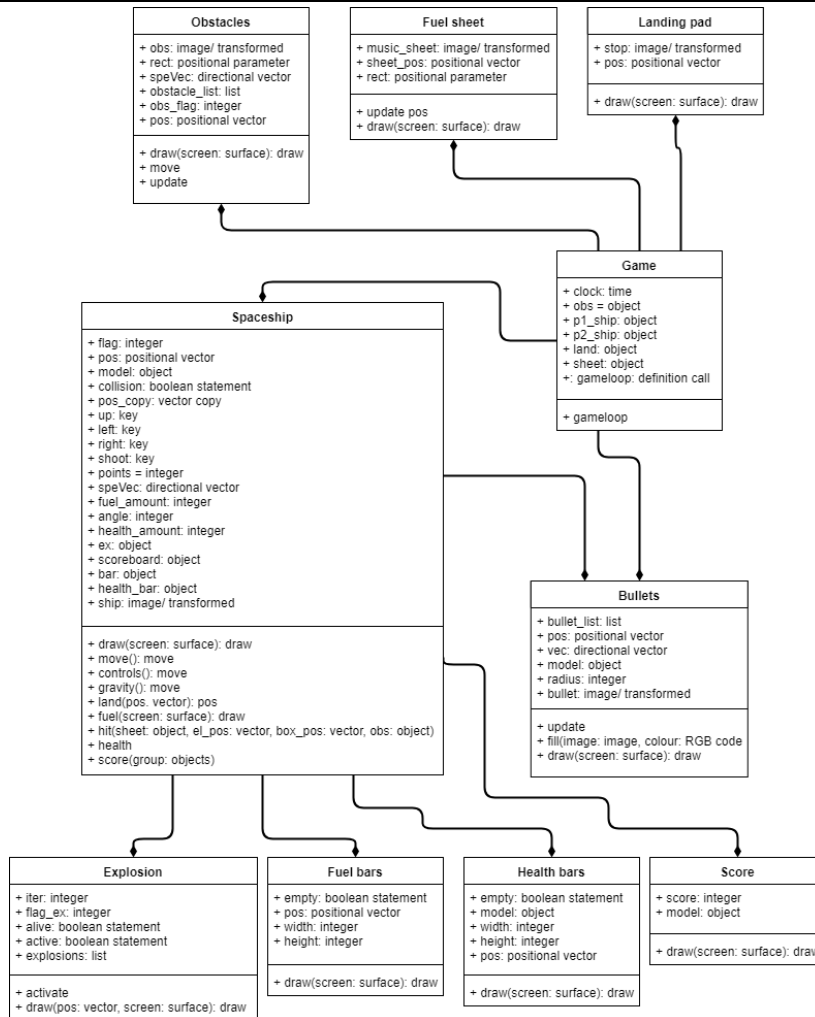
4 Implementation

The spaceship class is the largest and contains information about player events and actions. Its initializer receives various parameters such as the model, position, image, size, and controls for steering the spaceship. The class includes several methods:

- **Draw:** Checks the ship's life state and stops drawing its image if it dies.
- **Move:** Handles rotation of the ship's image based on user controls and updates its position. If the ship moves outside the screen boundaries, it explodes, causing the player to lose points.
- **Controls:** Manages player input for thrust and shooting actions.
- **Gravity:** Adds a downward-facing vector to the ship's position to simulate gravity.
- **Land:** Checks if the ship's radius connects with the stand on the screen to refuel.
- **Hit:** Detects collisions with obstacles and updates health and score accordingly.
- **Fuel:** Displays the remaining fuel and updates it as the game progresses.
- **Health:** Manages the health bar and checks if the player is still alive.
- **Score:** Updates the score based on game events.

The game class instantiates the remaining classes, sets up the screen and background, runs the game loop, and updates the display. It draws all objects, iterates through spaceship methods, updates and draws bullets, and manages game settings such as FPS and exit options.

The UML diagram illustrates how the spaceship and game classes instantiate other classes, demonstrating the overall structure of the project. Additionally, a configuration file contains constants used throughout the code, providing easy access and management of these values.



The UML diagram illustrates how the spaceship and game classes instantiate other classes, demonstrating the overall structure of the project. Additionally, a configuration file contains constants used throughout the code, providing easy access and management of these values.

5 Discussion

The alteration to convert all interactable objects in the game into `sprite.Sprite` and then place them into a group led to some issues. While it was relatively straightforward to adapt them to run, remodeling the interactions became necessary, as the previous method relied heavily on object lists for interactions. Additionally, there was some back-and-forth regarding interaction detection methods in general. Up until delivery, two of the interactions were more troublesome than the others. The ship-on-ship interaction and the bullet-on-ship interaction didn't detect properly for a while, however increasing the detection radius and creating two death flags resolved the issue last minute.

6 Conclusion

All the requirements for the assignment have been successfully met. The spaceships move independently and can lose points through crashes or destruction, while also gaining points for destroying the opponent.

7 References

The pygame library with its many functions included: pygame docs. 2018. *pygame documentation*. [ONLINE] Available at: <https://pygame.org/docs/>. [Accessed 20 April 2018].

The tool to make an UML-diagram:

draw. 2018. *Mayhem_UML*. [ONLINE] Available at: <https://www.draw.io/>. [Accessed 19 April 2018].

The book of this course:

Philips, D., 2015. *Python 3 Object-oriented Programming*. 2nd ed. Birmingham: Packt Publishing Ltd.